Query-Anchored Dense Community Search in Directed Citation Networks

The-Anh Vu-Le

Abstract

Paper-centered literature search asks for a compact community around a query paper, not only a ranked list of neighbors. We formulate this task as query-anchored dense community search in a directed citation graph. Given a query paper q and a minimum size k, the objective is to return a set $S \ni q$ that maximizes induced directed pair density $|E(S)|/(|S|(|S|-1))$, with an optional rooted edge-connectivity constraint from q.

Each oracle call reveals outgoing references and a capped number of incoming citations, so optimization and graph materialization proceed together. The solver combines Dinkelbach's fractional transform with lazy branch-and-price over a growing active set, with frontier pricing, branch-and-bound, triangle cuts, max-flow connectivity cuts, and incumbent pruning. We compare this objective against BFS, query-anchored average-degree, and grow-to-k baselines.

On Cora-ML, branch-and-price returns denser and more compact communities than the baselines. At $k = 3$, its edge density is 0.51 ± 0.10, against 0.35 ± 0.11 for depth-1 BFS and 0.21 ± 0.12 for AvgDeg. Depth-1 BFS remains the strongest classifier on the full Cora-ML test split, while dense-community search performs best on the subset where the direct BFS vote fails. The two findings separate cheap local classification from retrieval under explicit density, size, and connectivity constraints.

1 Introduction

A paper's outgoing citations identify the prior work it builds on, and its incoming citations identify the later work that builds on it. Most scholarly interfaces present these relationships as ranked lists or shallow neighborhoods. Neither view answers a structural question that often matters more to a reader: starting from a query paper, which nearby papers form a cohesive community?

A query paper may cite a broad survey, a standard dataset paper, or a method paper used across unrelated areas. Expanding from such a query yields a heterogeneous neighborhood with many individually relevant papers but little internal cohesion. A focused line of work, by contrast, appears as a compact set of papers with many citation edges among themselves. A dense induced subgraph is the natural abstraction for the second case: a group whose members are close to the query and to one another.

In this paper, we study dense community search under two constraints required by paper-centered retrieval. First, the answer must contain the query. Note that global dense-subgraph

algorithms can return an unrelated clique-like component elsewhere in the graph, which is unhelpful to a user starting from a particular paper. Second, the answer must satisfy $|S| \geq k$. Without a size floor, a density objective collapses to a trivial pair or a small motif rather than a reading-list-sized community.

A third constraint is operational. In live literature search, the full citation graph is not held in memory. The system queries a bibliographic index for a paper's references and, up to a configured cap, its citing papers. The algorithm therefore solves an information-acquisition problem alongside an optimization problem: it must decide which frontier papers are worth materializing. Branch-and-price addresses this requirement directly, since column generation acts as lazy graph expansion.

We do not propose this method as a replacement for one-hop BFS in every downstream task. Indeed, in our classification experiment, depth-1 BFS is the strongest full-test classifier, since the pure-source query pool makes direct citations to training-labelled papers highly informative. What we contribute is a retrieval primitive for compact, dense, query-containing citation communities, together with the controlled comparison needed to interpret that primitive.

Contributions. We make four contributions in this paper.

1. We formulate directed query-anchored dense community search with a minimum-size constraint and optional rooted κ-edge-connectivity.
2. We present a lazy branch-and-price algorithm for this problem, combining the Dinkelbach outer loop, an active-set restricted master problem, frontier pricing, branch-and-bound, triangle cuts, connectivity cuts, and incumbent pruning.
3. We describe BFS, average-degree, and grow-to-k baselines in the same formal language as our proposed method, clarifying what each baseline does and does not optimize.
4. We evaluate intrinsic cluster quality, computational cost, and downstream classification behavior on synthetic correctness tests and Cora-ML citation data.

Related work. Our work builds on classical dense-subgraph optimization. Goldberg [8] gave a flow-based algorithm for maximum-density (average-degree) subgraph, and Charikar [5] provided a simple greedy approximation viewpoint. Size-constrained and connected variants are more difficult and have motivated exact and approximation algorithms for densest k-subgraph and related problems [1, 4, 12]. Our setting differs in three ways: it is query anchored, directed, and locally materialized through an oracle.

We handle the fractional objective using Dinkelbach's transform [6], and we solve the inner parametric problem with a branch-and-price method in the sense of Barnhart et al. [2]: a restricted master problem is solved over a subset of columns, and pricing identifies variables that should enter. Our Boolean-quadric relaxation and its triangle inequalities draw on classical work on the Boolean quadric polytope [10] and product linearization [9]. We obtain citation data either from local CitationFull datasets, as exposed by PyTorch Geometric [3, 7], or from OpenAlex, a large open index of scholarly works [11].

2 Model and Algorithms

In this section, we develop the mathematical formulation and the algorithms used in the experiments. We first define the query-anchored objective and the oracle model. We then present the branch-and-price method, its optional rooted-connectivity mechanism, and the baseline procedures against which we compare it.

2.1 Problem and oracle model

Let $G = (V, E)$ be a directed graph without self-loops. In the citation setting, an edge $u \to v$ means that paper u cites paper v. For a node set $S \subseteq V$, let

$$E(S) = \{(u, v) \in E : u \in S,\ v \in S\}$$

be the directed edge set induced by S. The directed pair density of S is

$$\mathrm{dens}(S) = \frac{|E(S)|}{|S|(|S| - 1)}. \tag{1}$$

The denominator is the number of possible directed non-self edges on $|S|$ nodes, so $\mathrm{dens}(S) \in [0, 1]$.

Given a query node q and a minimum size k, the primary problem is

$$\begin{aligned} \max_{S \subseteq V} \quad & \mathrm{dens}(S) \\ \text{s.t.} \quad & q \in S, \qquad |S| \geq k. \end{aligned} \tag{2}$$

Note that we require $|S| \geq k$ rather than $|S| = k$. When the query lies in a natural community of size slightly larger than k, an equality constraint would discard relevant papers. At the same time, pair density discourages unnecessary growth: adding a node to a set of size n creates $2n$ additional admissible directed pairs that must be justified by induced citation edges.

We access the graph through an oracle $\mathcal{O}$. A call $\mathcal{O}(v)$ returns two lists: the predecessors of v (incoming citations) and the successors of v (outgoing references). Both the simulation oracle and the OpenAlex oracle return all outgoing references exposed by the data source and at most `max_in_edges` incoming citations; repeated calls are cached. This cap defines the revealed graph. When `max_in_edges`=0, incoming expansion is disabled, although an incoming edge can still be discovered when another queried paper exposes it as an outgoing reference.

Our solver maintains a revealed directed graph $G_R = (V_R, E_R)$ together with two nested search universes: an active set represented in the current restricted master problem, and a frontier of discovered nodes not yet promoted to that master problem. Optimization is therefore conditional on revelation. The active-set MILP is exact for the variables it contains, while frontier expansion is a lazy search over the latent graph unless the candidate universe has been fully materialized and no priced set improves the relaxation. This distinction underlies our empirical interpretation: the brute-force tests evaluate exact recovery on fully enumerable instances, and the Cora-ML experiments evaluate an oracle-limited search procedure.

2.2 Fractional objective and restricted master problem

The density objective in (2) is a ratio. For a fixed density estimate λ, we apply Dinkelbach's transform to obtain the parametric objective

$$\Phi_\lambda(S) = |E(S)| - \lambda|S|(|S|-1). \tag{3}$$

If some feasible set S has $\Phi_\lambda(S) > 0$, then its density exceeds λ; if the maximum value is zero, then no feasible set improves on λ for the current graph. We therefore alternate between solving the parametric problem and updating λ to the density of the returned set, and we stop when either the new parametric value falls below a tolerance ε or the new density fails to exceed λ by more than ε. We solve the LP relaxation in each parametric subproblem with the dual simplex method.

For a fixed active node set $\mathcal{A}$, we formulate the parametric problem with node variables x_v, directed-edge variables y_{uv}, and unordered-pair variables w_{uv}. Variable x_v indicates whether v is selected. For each materialized directed edge (u, v) with both endpoints active, y_{uv} represents $x_u x_v$ and contributes one unit of edge reward. For each unordered pair $\{u, v\} \subseteq \mathcal{A}$, w_{uv} represents $x_u x_v$ and contributes the density penalty. Since

$$|S|(|S|-1) = 2 \sum_{\{u,v\} \subseteq S} 1,$$

the restricted objective at density λ is

$$\sum_{(u,v) \in E_R(\mathcal{A})} y_{uv} - 2\lambda \sum_{\{u,v\} \subseteq \mathcal{A}} w_{uv}. \tag{4}$$

The LP relaxation solved at a branch-and-bound node is

$$\begin{aligned}
\max \quad & \sum_{(u,v) \in E_R(\mathcal{A})} y_{uv} - 2\lambda \sum_{\{u,v\} \subseteq \mathcal{A}} w_{uv} \\
\text{s.t.} \quad & x_q = 1, \qquad \sum_{v \in \mathcal{A}} x_v \geq k, \\
& y_{uv} \leq x_u, \qquad y_{uv} \leq x_v && ((u,v) \in E_R(\mathcal{A})), \\
& w_{uv} \geq x_u + x_v - 1 && (\{u,v\} \subseteq \mathcal{A}), \\
& x_v = 1 \ (v \in V_1), \qquad x_v = 0 \ (v \in V_0), \\
& 0 \leq x, y, w \leq 1.
\end{aligned} \tag{5}$$

The sets V_1 and V_0 are the branch-and-bound decisions at the current node. The product constraints are intentionally one-sided and exploit objective signs. For an integer solution, y_{uv} is set to one exactly when both endpoints are selected because it has positive objective coefficient and is upper-bounded by both x_u and x_v. Likewise, w_{uv} is set to the smallest feasible value because it has negative objective coefficient; with binary x, the lower bound $w_{uv} \geq x_u + x_v - 1$ is enough to force $w_{uv} = 1$ when both endpoints are selected and $w_{uv} = 0$ otherwise. In the LP relaxation these one-sided constraints are deliberately weaker than the full McCormick envelope. They keep the model smaller, but they also make fractional bounds

optimistic; triangle cuts and branch-and-bound supply the remaining relaxation strength within the active set.

The restricted master is maintained incrementally. New x variables are added when nodes enter $\mathcal{A}$; new y variables are added when a pending revealed edge has both endpoints active; new w variables are added for pairs involving newly active nodes. When λ changes across Dinkelbach iterations, only the objective coefficient of existing w variables changes. When rooted connectivity is enabled, we also maintain zero-objective undirected support variables z_{uv} for materialized unordered pairs; these variables are used only by the lazy connectivity cuts described below.

2.3 Lazy branch-and-price solver

Our branch-and-price solver is the core algorithm. It combines three nested mechanisms: Dinkelbach updates over density estimates, branch-and-bound over integer node decisions, and column generation over frontier nodes. We now describe each component.

Active set and frontier. The active set $\mathcal{A}$ contains nodes represented in the current restricted master problem. The frontier $\mathcal{F}$ contains nodes that have been discovered through an oracle query but not yet promoted to $\mathcal{A}$. We initialize $\mathcal{A}$ by BFS from q: at each step we pop a node, query its adjacency, insert it into $\mathcal{A}$, and push its predecessors and successors onto the BFS queue, stopping when $|\mathcal{A}| \geq k$. By construction, every node inserted into $\mathcal{A}$ has been queried, so its adjacency is fully materialized before the first LP is solved and all directed edges among active nodes appear as y_{uv} candidates in the restricted master. When we later promote a frontier node, it becomes active and we materialize its adjacency at that point.

Pricing derivation. Let $\bar{x}$ be the current LP solution and let π be the dual value of the size constraint. For a frontier node f, we define its fractional internal degree into the active solution by

$$d_{\text{frac}}(f) = \sum_{u:(u,f)\in E_R} \bar{x}_u + \sum_{v:(f,v)\in E_R} \bar{x}_v, \qquad u, v \in \mathcal{A}. \tag{6}$$

Both incoming and outgoing active neighbors count, since the objective counts directed induced edges. If we were to add f as a one-node probe with $x_f = 1$ while holding the active fractional solution fixed, then f would gain approximately $d_{\text{frac}}(f)$ in directed-edge reward, would incur a pair penalty of $2\lambda \sum_{v\in\mathcal{A}} \bar{x}_v$ against the active fractional mass, and would interact with the size-constraint dual. These three terms yield the reduced-cost score

$$\text{rc}(f) = d_{\text{frac}}(f) - 2\lambda \sum_{v\in\mathcal{A}} \bar{x}_v - \pi. \tag{7}$$

A positive score indicates that promoting f can improve the restricted LP. We promote a batch of the top positive frontier nodes rather than a single node, controlled by configurable minimum, maximum, and fractional batch-size parameters; in our experiments we use a fractional batch size of 1.0 with a hard cap of 50 per round.

Note that solo pricing is only a first-order test. A group of frontier nodes may have weak individual scores but strong mutual edges, so that each node appears unprofitable alone even though the group completes a dense block. We address this failure mode in two stages. First, when solo pricing stalls and unqueried frontier nodes remain, we materialize their adjacencies so that hidden frontier-frontier edges become visible, and we restart pricing. Second, once no further materialization is possible within the round, we invoke a bounded joint-pricing heuristic over $\mathcal{A} \cup \mathcal{F}$: when the number of combinations is within a budget of 5×10^5 query-containing k-sets we enumerate them exactly, and otherwise we greedily grow from $\{q\}$ by a cheap directed gain rule. This procedure improves recovery of clique-completer sets but remains a heuristic pricing oracle. A non-positive solo score combined with a failed bounded joint pass therefore certifies only that the pricing routine found no improving subset of $\mathcal{A} \cup \mathcal{F}$; it makes no claim over the unrevealed graph. We also expose a flag (`-no-materialize`) that disables this materialization step for expensive oracles, at the cost of reduced recall on frontier-internal cliques. The exhaustive joint-pricing branch scores full directed internal edge count. The greedy fallback scores each candidate by outgoing edges from the candidate into S, then breaks ties first by outgoing edges into the remaining pool and then by ascending node id.

Branching and cuts. Once column generation stalls, we use the LP solution for bounding. When the solution is integral, we treat the selected set as an incumbent candidate. When it is fractional, we select exactly one branching variable per LP node according to a 2λ internal-degree rule. If at least one fractional node has internal degree below 2λ, we call it a *hanging* node and branch on the hanging node with the smallest internal degree, with ties broken by closeness of $\bar{x}_v$ to 0.5, and we explore the $x_v = 0$ child first. Otherwise every fractional node has internal degree at or above 2λ, a *core* configuration, and we branch on the most fractional core node, with ties broken by larger internal degree, and we explore the $x_v = 1$ child first. The threshold follows the same marginal calculation as pricing: a selected node must contribute enough directed internal edge mass to offset the pair penalty induced by density λ.

We strengthen the relaxation with triangle inequalities for the Boolean quadric terms:

$$x_u + x_v + x_w - w_{uv} - w_{vw} - w_{uw} \leq 1. \tag{8}$$

These inequalities cut fractional patterns in which several x variables carry moderate mass but their pair variables remain too small. We separate them on fractional triples whose $\bar{x}$ values lie in $(0.1, 0.9)$, retain only cuts violated by more than 10^{-4}, and add at most 20 cuts per round. We further gate separation: we attempt it only when at least three fractional nodes remain, fewer than two consecutive LP-bound stalls have occurred, and the relative gap between the LP bound and the incumbent exceeds 1%. Together, these conditions strengthen the LP without dominating runtime.

Incumbent pruning. We greedily prune integer candidates and the initial BFS seed. The pruning rule removes a non-query node when doing so improves the relevant objective: pair density for the seed, or the current parametric value for branch-and-bound incumbents. We stop when either a removal would no longer improve the objective or the set has shrunk to

$|S| = k$, so that the minimum-size constraint remains a hard floor on the pruning loop. The least useful node is the one with smallest internal incident degree, since its removal loses the fewest induced edge contributions while reducing the pair penalty. When connectivity is active, we apply a local support-connectivity check that runs a single-source BFS from q over the induced support of $S \setminus \{u\}$, where u is the candidate for removal; we reject the removal when any other selected node becomes unreachable. If a post-prune κ verification still fails, we revert to the unpruned candidate.

Algorithm 1 summarizes the procedure. The pseudocode abstracts away low-level model synchronization and timing controls, but it preserves the search logic used in the experiments.

Algorithm 1: Lazy Dinkelbach branch-and-price

Input: query node q; minimum size k; oracle $\mathcal{O}$; tolerance ε
Output: best query-containing set found

```
(A, F) ← query-centered BFS initialization from q
S_0 ← greedy-pruned seed set; λ ← dens(S_0)
repeat
    initialize a depth-first branch-and-bound tree with root fix x_q = 1
    incumbent S^inc ← S_0
    while there is an unexplored branch-and-bound node do
        repeat
            synchronize the master with the current active set A
            read node fixes (V_1, V_0) and apply them to the restricted master
            solve the LP relaxation and obtain (x̄, π) and an LP bound
            compute rc(f) for all f ∈ F using (7)
            if any f ∈ F has rc(f) > 0 then
                promote a bounded batch of positive-reduced-cost frontier nodes and restart
            if some frontier node has unqueried adjacency then
                materialize all unqueried frontier adjacencies and restart
            if joint pricing has not yet been called at this CG node then
                invoke bounded joint pricing over A ∪ F; promote and restart on any
                  returned set
            if the gating conditions for triangle cuts hold then
                separate a bounded batch of violated triangle cuts and restart on success
        until no column or cut is added during the iteration
        if the LP bound cannot improve S^inc then
            prune the node and continue to the next branch-and-bound node
        if x̄ is integral (no fractional x̄_v) then
            S ← {v : x̄_v > 1/2}
            if κ > 0 and S violates rooted connectivity then
                add a lazy min-cut inequality, push the same branch-and-bound node back,
                  and continue
            greedily prune non-query vertices that improve Φ_λ, with revert when the prune
              breaks κ-edge-connectivity
            if Φ_λ(S) > Φ_λ(S^inc) then
                S^inc ← S and reset the no-improvement clock
        else
            choose one branching variable v by the 2λ internal-degree rule
            push the non-preferred child onto the depth-first stack first, then the preferred
              child, so that the preferred child is explored next
    if Φ_λ(S^inc) ≤ ε or dens(S^inc) ≤ λ + ε then
        break
    λ ← dens(S^inc) and S_0 ← S^inc
until the incumbent has nonpositive parametric value, or another stopping rule fires
return S_0
```

2.4 Rooted connectivity

The optional parameter κ supplies a robustness notion beyond density. Let $H(S)$ denote the undirected support graph obtained from the selected internal citation edges. The rooted condition requires that every $t \in S \setminus \{q\}$ admits at least κ edge-disjoint paths from q to t in $H(S)$. By max-flow/min-cut duality, this is equivalent to every q-t cut having capacity at least κ.

We enforce this condition lazily. For each materialized unordered pair $\{u, v\}$ with at least one directed citation arc, the solver creates a support variable z_{uv} satisfying

$$z_{uv} \leq x_u, \qquad z_{uv} \leq x_v, \qquad z_{uv} \leq y_{uv} + y_{vu},$$

omitting nonexistent directed arcs from the last sum. Thus z_{uv} can count at most one unit of undirected support capacity even when the citation pair is reciprocal.

When an integer candidate is produced, we construct a unit-capacity support graph and run a max-flow from q to each selected target t. When the flow falls below κ, the residual graph identifies a violated cut. Let R be the source side containing q. We add a cut of the form

$$\sum_{\{u,v\} \in \delta(R)} z_{uv} \geq \kappa x_t, \tag{9}$$

where $\delta(R)$ denotes the materialized support pairs crossing from R to its complement. We then reject the candidate and reprocess the node with the added inequality. This is standard lazy separation: we generate connectivity constraints only when needed, avoiding a large static family of cuts. Because the active set can grow after a cut is generated, the implementation records each cut's source side and adds newly materialized crossing support variables to the existing inequality as they enter the restricted master.

2.5 Optimality, revelation, and stopping rules

We distinguish three notions of optimality for our solver. First, for a fixed active set and a fixed λ, branch-and-bound with the current linearization and all generated cuts solves the active-set integer problem up to numerical tolerance, provided no limits interrupt it. Second, column generation expands the active set through a pricing routine; solo reduced-cost pricing is exact only for one-node additions, while materialization and joint-pricing are practical safeguards against missed frontier cliques rather than a complete subset-pricing oracle. Third, the live citation graph may contain unrevealed nodes, since incoming citations are capped and API queries are issued only on demand. We therefore describe our method as an exact active-set optimizer embedded within a heuristic, oracle-limited graph-expansion process.

Our two time limits reflect the same distinction. The soft limit is not a wall-clock cap on the entire solver: it is a no-improvement budget inside the branch-and-bound search, with the accumulated oracle network time subtracted from this clock and the clock reset every time a new incumbent is accepted. It also does not fire until an incumbent with positive parametric value exists, since before that point the Dinkelbach outer loop relies on the subproblem search to determine whether the current density estimate is already unimprovable. The hard limit is a separate wall-clock cap over the whole branch-and-price solve; when it fires, we

return the best incumbent found so far. In addition to the two time limits, the solver accepts optional bounds on branch-and-bound node count (`-node-limit`), Dinkelbach iterations (`-dinkelbach-iter`), and a heuristic upper-bound gap (`-gap-tol`) that prunes the tree once the relative gap between the per-iteration maintained UB and the current incumbent falls below the tolerance. All three are disabled by default and remain so in our experiments.

These stopping rules determine the strength of the empirical claims we can make. A completed no-limit run on a small fully materialized graph supports an optimality claim. A time-capped Cora-ML run supports a claim about performance and retrieval quality under the stated oracle budget, but not a global optimality claim over the full citation graph.

2.6 Baselines and post-processing

We use three baselines to define the comparative meaning of our proposed method. All baselines share the oracle conventions, including the incoming-edge cap and caching, but they differ in their objective and in whether they optimize a structural criterion at all.

BFS neighborhood extractor. Our BFS baseline performs a layer-by-layer expansion from q. We query each node in the current frontier and record its predecessors and successors in an undirected adjacency map for expansion. For a depth parameter d, the returned set is the closed ball of radius d under this expansion convention. We also query the last layer, so that nodes at distance $d+1$ can enter the adjacency cache as candidates for grow-to-k, even though they are not part of the native BFS output. This baseline carries no density objective and no connectivity optimization. When a target size k is supplied and the BFS set has fewer than k nodes, we invoke the grow-to-k procedure described below.

Algorithm 2: BFS neighborhood baseline

Input: query node q; oracle $\mathcal{O}$; depth d; optional target size k
Output: BFS neighborhood, optionally grown to size k

```
S ← {q}; F ← {q}; mark q as visited
initialize an undirected adjacency cache A and empty queried/error sets
for ℓ = 0, 1, ..., d while F ≠ ∅ do
    F⁺ ← ∅
    foreach unqueried non-error vertex u ∈ F do
        query O(u) for predecessors and successors
        record both directions in A for expansion
        append newly discovered vertices to F⁺
    if ℓ < d then
        S ← S ∪ F⁺
    F ← F⁺
if k is supplied and |S| < k then
    S ← GrowToK(S, A, O, k)
return S
```

Average-degree solver. Our average-degree baseline optimizes a different density objective:

$$\text{avgdens}(S) = \frac{|E(S)|}{|S|}, \qquad S \ni q, \qquad S \subseteq V_{\text{loc}}. \tag{10}$$

We first materialize a local graph around q by BFS under the same oracle convention as the other methods. Note that in the experiments below the exploration depth is unbounded within the graph exposed by the oracle, so AvgDeg is not a depth-one baseline. We build the materialized directed edge list from the outgoing successor lists of queried nodes; predecessor information expands the search frontier, while we count a directed internal edge whenever its source has been queried and its target also lies in the local graph.

For a candidate threshold λ, AvgDeg solves

$$\begin{aligned} \max_{S} \quad & |E(S)| - \lambda |S| \\ \text{s.t.} \quad & q \in S, \qquad S \subseteq V_{\text{loc}}. \end{aligned} \tag{11}$$

using a Goldberg-style max-flow construction. We represent each directed edge by an item node with a source arc of capacity $\texttt{SCALE} = 10^6$; we connect each item node to its two endpoints with infinite capacity, connect each paper node to the sink with capacity $\text{round}(\lambda \cdot \texttt{SCALE})$ rounded to the nearest non-negative integer, and force q onto the source side with an infinite-capacity source arc. The source-side residual set after max-flow gives the query-anchored threshold optimum. We bisect λ for up to 100 iterations with bracket tolerance 10^{-9}, then probe the final bracket endpoints, and keep the best observed $|E(S)|/|S|$ solution, so that exactness holds up to the materialized graph, integer scaling, and bisection tolerance.

This baseline is exact for its materialized local average-degree objective up to numerical tolerance, but that objective is not pair density. AvgDeg tends to prefer larger sets, because adding nodes can sustain the average internal degree even when the fraction of admissible directed pairs declines. Our empirical tables reflect this behavior: AvgDeg often achieves the highest average internal degree but substantially lower pair density and a much larger output size.

Algorithm 3: Query-anchored average-degree baseline

Input: query node q; oracle $\mathcal{O}$; exploration depth d; optional target size k
Output: query-anchored set maximizing local $|E(S)|/|S|$, optionally grown to k

```
materialize a local graph G_loc = (V_loc, E_loc) by BFS from q to depth d (or all reachable
  nodes if d < 0)
[L, U] ← [0, |E_loc|]; S* ← {q}
for at most the bisection iteration limit do
    λ ← (L + U)/2
    build the threshold network with one edge-item node per directed edge
    force q onto the source side with an infinite-capacity source arc
    run max flow and let S_λ be the source-side paper vertices
    if |E(S_λ)| − λ|S_λ| > 0 then
        L ← λ
    else
        U ← λ
    if |E(S_λ)|/|S_λ| improves the incumbent then
        S* ← S_λ
probe the final bracket endpoints and keep the best observed set
if k is supplied and |S*| < k then
    S* ← GrowToK(S*, A, O, k)
return S*
```

Grow-to-k. Neither BFS nor AvgDeg inherently solves an at-least-k pair-density problem. To compare them at the same nominal sizes, we apply a deterministic grow-to-k heuristic whenever a baseline returns fewer than k nodes. Given a seed set S and a pool of discovered candidates, the heuristic repeatedly adds

$$v^\star = \arg\max_{v \notin S} |N(v) \cap S|, \tag{12}$$

breaking ties by node id. When the candidate pool is empty but some selected node has not yet been queried, we query the oracle to expose more candidates. We chose this procedure to be simple and deterministic. It is not an approximation algorithm for densest-at-least-k; its role is to prevent baselines from being unfairly small in per-k comparisons. Because we query newly selected candidates, grow-to-k can also reveal additional local structure, but the selection rule remains myopic: each step maximizes current adjacency to S, not the final pair-density objective.

Algorithm 4: Grow-to-k post-processing

Input: seed set S; oracle $\mathcal{O}$; adjacency cache A; queried/error sets; target size k
Output: a deterministic superset of the seed, if enough candidates are reachable

```
C ← {v ∉ S : ∃u ∈ S with v ∈ A(u)}
while |S| < k do
    if C = ∅ then
        choose an unqueried non-error probe vertex u ∈ S
        if no such u exists then
            return S
        query O(u), update A, and add new non-selected neighbors to C
    choose v* ∈ C maximizing |A(v) ∩ S|; break ties by node id
    S ← S ∪ {v*} and C ← C \ {v*}
    if v* has not been queried then
        query it and add newly discovered non-selected neighbors to C
return S
```

3 Experimental Design

We organize our evaluation around three questions. First, does branch-and-price recover the known optimum on graphs small enough to brute force? Second, on a real citation graph, what intrinsic community geometry does it return, compared with BFS and AvgDeg? Third, when we use the returned communities for a simple label-propagation readout, how much semantic information do they preserve?

3.1 Datasets, splits, and methods

We use Cora-ML from the CitationFull family as our main dataset. It contains 2,995 papers, 8,416 directed citation edges, and seven labels. We apply an inductive split that isolates queries from their own labels. The query pool consists of the *pure sources*: papers that cite other papers but are not themselves cited in the exported graph. We then filter to out-degree at least two and split the resulting pool evenly into validation and test by shuffling under random seed 42; the remaining papers form the training pool. We use 402 validation queries and 402 test queries; the training pool contains 2,191 labeled papers. Query nodes from validation and test are never used as labels for one another.

We compare three methods: depth-1 BFS, query-anchored AvgDeg, and branch-and-price (BP), evaluated at $k \in \{3, 4, 5\}$ and $\kappa \in \{0, 1, 2\}$. We apply grow-to-k to BFS and AvgDeg only when their native output has fewer than k nodes. We run BP with a 60 second soft no-improvement cap and a 300 second hard wall-time cap in the cost and quality experiments. Recall from Section 2.5 that the soft cap is an algorithmic no-improvement budget that fires only after a positive incumbent exists, while the hard cap is a wall-clock bound on the solve. In these Cora-ML experiments we use the default incoming cap of zero, so that our main signal is the outgoing citation structure plus edges discovered through queried outgoing references. AvgDeg and BFS are deterministic and we run each one once per cell.

BP depends on Gurobi's random seed, which we fix to 42 throughout: in pilot experiments the across-seed spread of BP's pair density was small relative to the wall-time cost of repeating the cluster-quality and cost sweep on additional seeds. We select validation-best parameters on these single-seed records and report test-set confidence intervals through a paired bootstrap with $B = 500$ resamples on the per-query prediction indicators.

Our intrinsic quality metrics include output size, algebraic connectivity λ_2 of the normalized Laplacian on the undirected induced support, an internal min-normalized-cut robustness score, directed internal average degree, and directed pair density. We use the normalized-cut score as an internal robustness diagnostic: larger values indicate that the induced support is harder to split by a sparse internal cut. We compute directed average degree and directed pair density from outgoing edge lists, matching the directed objective of our solver. Our cost metrics include wall time, oracle queries, branch-and-bound nodes, LP solves, cuts, and LP time. For reproducibility, we fix the Gurobi random seed via `-gurobi-seed` and record for each solver run the output size, oracle-query count, wall and LP-solve times, cut and branch-and-bound counts, hard-cap status, κ-verification flags, and the requested intrinsic quality metrics; we optionally persist the selected node list and the full Dinkelbach trajectory to a separate solver-dump file when full traceability is needed.

We classify each query by a uniform majority vote over the training-labeled members of the returned community, breaking ties by ascending label for reproducibility. When a returned community contains no training nodes, we apply a concentric BFS fallback over the directed citation graph that searches outward up to 10 hops while excluding validation and test nodes as stepping stones, votes a uniform majority over the nearest reachable training-labeled nodes, and falls back to the global training-majority label when no training label is reachable. In our Cora-ML records, this fallback path is never triggered: every returned community contains at least one training-labelled paper. We report macro-averaged precision, recall, and F1 throughout. We compute the hard subset independently of BP and of any solver: it consists of the test nodes whose true label disagrees with the majority label over their outgoing 1-hop training neighbours, with a global training-majority fallback when no training neighbour is reachable. We materialise this subset at data-preparation time directly from the citation graph and the training mask, and we store its hash in the split metadata. Note that this is a diagnostic subset for shallow-neighborhood failure, not a new primary benchmark.

3.2 Brute-force correctness protocol

We test correctness on synthetic graphs small enough for exhaustive enumeration. Our fixture generates undirected Erdős–Rényi graphs with $n = 20$ and per-pair edge probability $p \in \{0.25, 0.50, 0.75\}$, then converts each undirected edge into a reciprocal pair of directed arcs ($u \to v$ and $v \to u$) to match the directed convention used in the rest of the paper. Five seeds $(0, \ldots, 4)$ per p give 15 graphs. For each graph, brute force enumerates every subset containing the query node and records the best pair-density value for each $k \in \{3, 4, 5\}$ and $\kappa \in \{0, 1, 2\}$. The $\kappa = 0$ reference applies no connectivity filter; for $\kappa > 0$, we compute rooted edge-connectivity on the undirected support and keep only subsets meeting the requested threshold. We skip infeasible κ cells. We then run the solver on the same grid without limiting time, nodes, gap, or Dinkelbach iterations.

The purpose of this test is narrow but important. It does not imply polynomial-time

behavior or global optimality under oracle caps; rather, it checks that our restricted master, Dinkelbach updates, branch-and-bound, pricing and materialization logic, cut generation, and pruning agree with an independent brute-force reference on small fully materialized instances. In particular, it exercises the same mechanisms that later appear in the Cora-ML runs, while removing API incompleteness and time limits.

4 Results

4.1 Correctness on enumerable graphs

On the enumerable test grid, we observe that AvgDeg matches the brute-force average-degree optimum on all 15 graphs. BP matches the brute-force pair-density value in all 135 cells, and it satisfies $|S| \geq k$ throughout. For $\kappa > 0$, every returned BP set also satisfies the requested rooted edge-connectivity threshold. We observe no solver errors, hard-cap hits, or κ verification failures. Some cells have multiple optimal communities with the same density, so the solver's returned node set need not match the particular brute-force representative. For example, at $n = 20$, $p = 0.5$, seed 4, $k = 3$, BP returns a 4-node clique while the brute-force record stores a 3-node clique; both have pair density 1.0, and the returned set has rooted edge-connectivity 3. We interpret this result as a validation of the optimization pipeline: our Dinkelbach updates, active-set formulation, lazy materialization and pricing, branch-and-bound, connectivity separation, triangle cuts, and pruning agree with an independent exhaustive reference on the only instances where the full feasible region is small enough to enumerate.

We do not present this test as a scalability claim. Its value lies in isolating algorithmic correctness from dataset effects. On Cora-ML, a method may fail to recover a globally best community because the instance is hard, because the graph is only partially revealed through the oracle, or because time limits stop the search. The brute-force fixture removes those complications: we materialize the graph in full, disable all limits, and check the objective value exactly. Passing this fixture supports the later interpretation that differences among BFS, AvgDeg, and BP reflect their objectives and constraints rather than a defect in our active-set optimizer.

4.2 Intrinsic community quality

Table 1 reports cluster quality on the combined validation and test pools (804 queries). Each method recovers what it optimizes. BP, which optimizes directed pair density, returns the highest pair-density communities in every k block. AvgDeg, which optimizes directed edges per selected node, attains the highest average internal degree but substantially lower pair density. BFS optimizes neither objective, and its cohesion is incidental to a reachability rule.

Table 1: Cluster quality on Cora-ML validation+test queries (804 queries), mean ± standard deviation. The best value within each k block is underlined; the best value across all k values is bold. Higher is better for λ_2, the internal normalized-cut robustness score, avg.deg, and density; size is descriptive.

Method	$\lvert S\rvert$	λ_2	ncut	avg.deg	dens.
$k = 3$					
BFS $d = 1$	4.87 ± 2.75	0.96 ± 0.30	1.23 ± 0.15	1.17 ± 0.50	0.35 ± 0.11
AvgDeg	21.20 ± 15.20	0.32 ± 0.38	0.70 ± 0.53	$\underline{3.02 \pm 1.55}$	0.21 ± 0.12
BP $\kappa = 0$	3.34 ± 0.71	1.23 ± 0.34	1.36 ± 0.26	1.20 ± 0.42	$\underline{0.51 \pm 0.10}$
BP $\kappa = 1$	3.32 ± 0.67	1.25 ± 0.28	1.39 ± 0.13	1.19 ± 0.41	$\underline{0.51 \pm 0.10}$
BP $\kappa = 2$	3.28 ± 0.72	$\underline{1.27 \pm 0.27}$	$\underline{1.41 \pm 0.11}$	1.10 ± 0.44	0.48 ± 0.12
$k = 4$					
BFS $d = 1$	5.20 ± 2.56	0.78 ± 0.25	1.17 ± 0.09	1.22 ± 0.47	0.32 ± 0.10
AvgDeg	21.21 ± 15.18	0.31 ± 0.35	0.69 ± 0.53	$\underline{3.02 \pm 1.55}$	0.21 ± 0.11
BP $\kappa = 0$	4.35 ± 0.78	0.75 ± 0.48	0.94 ± 0.53	1.59 ± 0.48	$\underline{0.47 \pm 0.09}$
BP $\kappa = 1$	4.23 ± 0.76	0.92 ± 0.28	1.21 ± 0.10	1.47 ± 0.42	0.46 ± 0.09
BP $\kappa = 2$	4.19 ± 0.72	$\underline{0.94 \pm 0.28}$	$\underline{1.24 \pm 0.08}$	1.34 ± 0.46	0.42 ± 0.12
$k = 5$					
BFS $d = 1$	5.72 ± 2.36	0.66 ± 0.24	1.13 ± 0.08	1.29 ± 0.46	0.29 ± 0.09
AvgDeg	21.25 ± 15.14	0.30 ± 0.34	0.69 ± 0.52	$\underline{3.02 \pm 1.55}$	0.20 ± 0.11
BP $\kappa = 0$	5.03 ± 0.72	0.50 ± 0.44	0.74 ± 0.57	1.76 ± 0.51	$\underline{0.43 \pm 0.09}$
BP $\kappa = 1$	5.14 ± 1.03	$\underline{0.73 \pm 0.27}$	1.14 ± 0.08	1.67 ± 0.47	0.41 ± 0.09
BP $\kappa = 2$	5.16 ± 1.13	0.72 ± 0.28	$\underline{1.16 \pm 0.08}$	1.52 ± 0.54	0.37 ± 0.11

The contrast between AvgDeg and BP is informative. At $k = 3$, AvgDeg has average degree 3.02 ± 1.55, the highest value in the block, but pair density of only 0.21 ± 0.12. BP at $\kappa = 0$ has lower average degree, 1.20 ± 0.42, and higher pair density, 0.51 ± 0.10. Note that the two metrics differ in their denominator: average degree divides edges by $|S|$ and rewards large sets that maintain a constant per-node edge contribution, while pair density divides by $|S|(|S| - 1)$ and penalizes diffuse expansion. AvgDeg therefore collects broad citation regions with many internal edges in absolute terms, while BP extracts a smaller core in which a larger fraction of admissible directed pairs is realized.

We also observe that as k grows from 3 to 5, BP's pair density at $\kappa = 0$ falls from 0.51 to 0.43. The denominator grows quadratically with each additional required paper, and Cora-ML citation neighborhoods rarely contribute enough new internal edges to compensate. BP responds by keeping the returned set close to the requested minimum, while AvgDeg remains near twenty-one nodes across all k, since its unconstrained optimum already exceeds the requested size.

Increasing κ improves the support-graph connectivity diagnostics and lowers pair density. For example, at $k = 4$, algebraic connectivity rises from 0.75 ± 0.48 at $\kappa = 0$ to 0.94 ± 0.28 at $\kappa = 2$, while pair density falls from 0.47 ± 0.09 to 0.42 ± 0.12. Rooted edge connectivity excludes dense pieces that are attached to the query only through narrow bottlenecks, at the cost of slightly lower density. In retrieval terms, $\kappa = 0$ returns the densest compact core, while larger κ returns a less dense but more cut-resilient community.

4.3 Computational cost

Table 2 reports cost on the same Cora-ML val+test runs. BFS finishes in tens of milliseconds, AvgDeg remains fast at this scale, and BP is the expensive method. The ordering tracks how

much search each method performs. BFS issues a small number of oracle calls and runs no optimization. AvgDeg materializes a larger local region and solves a sequence of max-flow problems. BP synchronizes the active set, solves repeated LP relaxations, prices, branches, separates triangle cuts, and, when $\kappa > 0$, also separates max-flow connectivity cuts.

Table 2: Cost on Cora-ML validation+test queries (804 queries), mean $\pm$ standard deviation. BP uses a 60 s soft no-improvement cap and a 300 s hard cap. "hc" is the fraction of queries on which the hard cap fired.

Method	wall (s)	queries	BB nodes	LP solves	cuts	t_{LP} (s)	hc
$k = 3$							
BFS $d = 1$	0.03 ± 0.01	5 ± 3	–	–	–	–	–
AvgDeg	0.08 ± 0.05	202 ± 210	–	–	–	–	–
BP $\kappa = 0$	1.54 ± 15.19	179 ± 201	121 ± 837	131 ± 861	87 ± 315	0.93 ± 11.50	0.001
BP $\kappa = 1$	1.47 ± 13.71	179 ± 201	121 ± 772	132 ± 796	90 ± 308	0.87 ± 10.4	0.000
BP $\kappa = 2$	10.3 ± 53.1	180 ± 202	81 ± 429	1106 ± 5876	1106 ± 5868	6.26 ± 36.3	0.032
$k = 4$							
BFS $d = 1$	0.03 ± 0.01	5 ± 3	–	–	–	–	–
AvgDeg	0.08 ± 0.05	202 ± 210	–	–	–	–	–
BP $\kappa = 0$	9.85 ± 38.12	200 ± 210	833 ± 2025	881 ± 2079	492 ± 677	9.21 ± 36.31	0.009
BP $\kappa = 1$	28.2 ± 79.7	200 ± 210	1549 ± 3593	1618 ± 3696	756 ± 1348	26.9 ± 76.8	0.070
BP $\kappa = 2$	47.9 ± 103.3	200 ± 210	1363 ± 3061	3794 ± 9691	3300 ± 9483	44.1 ± 95.6	0.133
$k = 5$							
BFS $d = 1$	0.04 ± 0.01	6 ± 2	–	–	–	–	–
AvgDeg	0.08 ± 0.05	202 ± 210	–	–	–	–	–
BP $\kappa = 0$	66.7 ± 110.0	202 ± 210	2967 ± 4275	3079 ± 4392	1291 ± 1477	63.2 ± 105.6	0.134
BP $\kappa = 1$	92.4 ± 130.6	202 ± 210	3799 ± 5311	3961 ± 5485	1992 ± 2825	88.4 ± 126.3	0.248
BP $\kappa = 2$	96.0 ± 131.8	202 ± 210	3197 ± 4493	4409 ± 7318	3326 ± 6744	90.8 ± 126.7	0.270

The mean BP wall time at $\kappa = 0$ rises from 1.54 seconds at $k = 3$ to 66.7 seconds at $k = 5$. Larger k enlarges the coordinated node set, introduces more pair variables, and makes fractional solutions harder to close in a small number of branches. The LP-solve counts and branch-and-bound node counts grow in the same direction, so the dominant cost is the combinatorial search over compact dense cores rather than oracle access.

Increasing κ acts through a different mechanism. The robustness constraint is inactive on most easy candidates but expensive once violated: each violating integer candidate triggers max-flow separation, adds a cut, and forces an LP resolve. At $k = 4$, moving from $\kappa = 0$ to $\kappa = 2$ raises the mean cut count from 492 to 3300 and the mean LP solves from 881 to 3794. The wall-time standard deviations are also large, indicating that the mean is driven by a long tail of difficult queries in which density, size, frontier discovery, and rooted connectivity fail to align on an obvious local core. The same tail manifests as hard-cap incidence: at $k = 5$, the hard cap fires on 13.4% of queries at $\kappa = 0$, rising to 24.8% at $\kappa = 1$ and 27.0% at $\kappa = 2$ (column hc in Table 2). Means in those cells are therefore lower bounds on the wall time that an uncapped solver would record.

Note that BP is not the appropriate tool when any neighborhood will do. Its purpose is to optimize a structural retrieval objective that BFS does not attempt. For fast topical preview, the cost table favors BFS; for an explicit density objective with a minimum size and optional connectivity, the additional LP and cut work is the cost of solving the harder problem.

The standard deviations also caution against treating the mean as a typical interactive

latency. Most queries solve quickly, while a smaller number trigger repeated LP resolves, frontier materialization, or connectivity cuts and dominate the average. Our stopping rules contribute to the spread as well: the hard cap can return an incumbent without proving optimality, and the soft cap measures time without incumbent improvement rather than total elapsed time. Structural quality and hard-cap status should therefore be read alongside wall time, not after it.

4.4 Classification readout

In this experiment, we ask whether the retrieved communities preserve label information. We tune each method on validation and then evaluate on test. Our prediction rule is intentionally simple: a majority vote over the training-labeled nodes in the returned community, with deterministic tie-breaking and the fallback described in Section 3.1. We chose this rule to isolate the effect of the retrieved subgraph rather than introducing a learned classifier. It also makes the limitations visible: a method can return an excellent structural community but still lose a classification vote if it filters out the most label-informative direct references.

Table 3 reports validation results for the tuned grid. BFS at depth 1 and $k = 4$ is the strongest validation classifier; among BP configurations, $k = 4, \kappa = 2$ is best. AvgDeg trails, since its larger output communities dilute the local label signal.

Table 3: Validation performance on Cora-ML (402 queries), mean ± bootstrap standard error (estimated as one quarter of the 95% paired-bootstrap CI width). Best within each k block is underlined; overall best is bold.

Method	Precision	Recall	F1
$k = 3$			
BFS $d = 1$	$\underline{0.836 \pm 0.019}$	$\underline{0.832 \pm 0.020}$	$\underline{0.826 \pm 0.021}$
AvgDeg	0.714 ± 0.026	0.540 ± 0.024	0.558 ± 0.026
BP $\kappa = 0$	0.797 ± 0.021	0.787 ± 0.022	0.782 ± 0.022
BP $\kappa = 1$	0.799 ± 0.020	0.790 ± 0.021	0.785 ± 0.021
BP $\kappa = 2$	0.801 ± 0.020	0.794 ± 0.021	0.785 ± 0.022
$k = 4$			
BFS $d = 1$	$\underline{0.844 \pm 0.019}$	$\underline{0.843 \pm 0.019}$	$\underline{0.838 \pm 0.020}$
AvgDeg	0.717 ± 0.026	0.544 ± 0.024	0.563 ± 0.025
BP $\kappa = 0$	0.762 ± 0.021	0.683 ± 0.025	0.694 ± 0.025
BP $\kappa = 1$	0.808 ± 0.020	0.789 ± 0.021	0.789 ± 0.022
BP $\kappa = 2$	0.827 ± 0.020	0.810 ± 0.021	0.811 ± 0.021
$k = 5$			
BFS $d = 1$	$\underline{0.814 \pm 0.021}$	$\underline{0.816 \pm 0.021}$	$\underline{0.810 \pm 0.021}$
AvgDeg	0.717 ± 0.026	0.544 ± 0.024	0.563 ± 0.025
BP $\kappa = 0$	0.725 ± 0.023	0.616 ± 0.026	0.628 ± 0.026
BP $\kappa = 1$	0.772 ± 0.024	0.744 ± 0.023	0.741 ± 0.024
BP $\kappa = 2$	0.795 ± 0.022	0.784 ± 0.021	0.780 ± 0.022

The validation table already indicates that classification and dense retrieval are not identical objectives. BP substantially improves pair density in Table 1, yet that improvement does not translate into the best label vote. Recall that our query pool restricts validation and test queries to pure sources, which by construction cite labelled training papers; in that setting, depth-1 BFS retrieves precisely the references whose labels are most predictive. BP filters this neighborhood to satisfy density and (when $\kappa > 0$) robustness, and the filter can

remove direct labeled neighbors that are useful for voting but weakly embedded in the dense core.

We evaluate the validation-best configurations on the test set in Table 4. BFS remains strongest, BP is second, and AvgDeg is third. The gap between BP and AvgDeg is informative. AvgDeg often includes more training papers, but a larger community can mix labels from multiple nearby topics. BP keeps the set close to the requested size and therefore preserves a sharper local vote, even though it does not match the direct-neighbor signal that BFS captures on the full split.

Table 4: Validation-best configurations evaluated on Cora-ML test queries (402 queries), mean ± bootstrap standard error (estimated as one quarter of the 95% paired-bootstrap CI width).

Method	validation-best parameters	Precision	Recall	F1
BFS	$d = 1, k = 4$	0.871 ± 0.016	0.853 ± 0.020	0.856 ± 0.019
AvgDeg	$k = 4$	0.772 ± 0.022	0.579 ± 0.024	0.603 ± 0.024
BP	$k = 4, \kappa = 2$	0.818 ± 0.021	0.815 ± 0.021	0.811 ± 0.021

We also examine a complementary subset: the 59 test nodes on which the outgoing depth-1 BFS vote is wrong by construction. Table 5 reports performance on this subset, where BP attains the highest F1. Note that this subset is conditional and should not be used as the primary benchmark; we use it diagnostically, to study what happens after the direct citation vote has already failed.

Table 5: Validation-best configurations on the 59 test nodes where the outgoing depth-1 BFS vote is wrong, mean ± bootstrap standard error (estimated as one quarter of the 95% paired-bootstrap CI width).

Method	validation-best parameters	Precision	Recall	F1
BFS	$d = 1, k = 4$	0.183 ± 0.061	0.097 ± 0.049	0.080 ± 0.042
AvgDeg	$k = 4$	0.269 ± 0.084	0.251 ± 0.079	0.146 ± 0.056
BP	$k = 4, \kappa = 2$	0.174 ± 0.053	0.282 ± 0.090	0.190 ± 0.056

We read this hard-subset result as clarifying when dense-community search helps. When direct neighbors already provide the correct label, BFS is difficult to beat with any filtered community. When direct neighbors point in the wrong direction, returning the same shallow neighborhood cannot fix the error. BP's advantage on the hard subset suggests that a dense, query-anchored core can recover signal that is not captured by the immediate citation vote. The precision and recall pattern is still modest and noisy, since the subset has only 59 queries, but the direction is informative: density optimization is most valuable when shallow reachability is misleading, ambiguous, or contaminated by bridge citations.

This readout suggests a two-mode deployment. A literature-search system can expose both a shallow-neighborhood mode and a dense-community mode: the former for fast label-propagation relevance and first-pass exploration, the latter for focused reading sets and for

identifying a cohesive structural context around a query paper. Disagreement between the two outputs is itself informative: it may indicate that the query is a bridge paper, a survey, or a paper whose references span multiple areas while one dense subcommunity dominates its local citation structure.

4.5 Interpretation of empirical trends

Each of the three methods returns a different graph object because each optimizes a different notion of locality. BFS returns a shallow reachability neighbourhood. AvgDeg returns a broad region with many internal edges per node. BP returns a compact set with high induced pair density and, when $\kappa > 0$, rooted edge-connectivity from the query. None of these objects is universally best; each is appropriate for a different retrieval question.

The computational trends follow from this algorithmic structure. BFS has low query counts because it stops after a fixed radius and performs no global optimization. AvgDeg has higher query counts because it materializes the local reachable graph before solving its flow sequence, and its optimization is polynomial and cheap at our scale. BP has similar or slightly lower oracle counts than AvgDeg in many cells but much higher wall time, since the dominant work is LP search rather than graph loading. The large cut and LP counts at $\kappa = 2$ show that rooted connectivity actively reshapes the search through lazy max-flow separation, rather than acting as a post-hoc filter.

On classification, structural cohesion and predictive label propagation can diverge. On the full Cora-ML test split, direct references to training papers are highly informative, and BFS is the best classifier on that split. On the BFS-hard subset, where this direct vote is known to fail, BP recovers more signal than the baselines. These results support a two-mode use of the system: BFS for fast topical expansion and label-propagation tasks, and dense-community search when the goal is a cohesive, size-controlled, query-centered research community.

5 Conclusion

Summary. In this paper, we formulated query-anchored dense community search for directed citation networks and developed a lazy optimization method for it. Our formulation asks for a query-containing set of at least k papers that maximizes induced directed pair density, with optional rooted edge-connectivity from the query. We solve this formulation with a branch-and-price method that combines Dinkelbach updates, an active-set restricted master problem, frontier pricing, branch-and-bound, triangle cuts, max-flow connectivity cuts, and incumbent pruning. Our method is an exact optimizer for the active-set subproblems, embedded within a practical oracle-limited expansion process. We compared it to BFS, AvgDeg, and grow-to-k baselines, which clarify the trade-off among reachability, average-degree density, pair density, size control, and computational cost.

Empirically, the method does what its objective prescribes: it returns compact communities with substantially higher pair density than BFS and AvgDeg on Cora-ML. It is also more expensive, particularly as k and κ increase. Our downstream classification results show that this structural gain does not translate into universal predictive superiority. Direct one-hop neighbours are the strongest signal on the full Cora-ML test split, since our query pool of pure

sources makes direct citations highly informative; dense-community search becomes more useful on the diagnostic subset where that direct vote fails. We therefore position our method as a retrieval primitive that returns a structurally distinct graph object, complementary to shallow reachability rather than a replacement for it.

Future work. This study suggests several directions for future research.

Tighter LP relaxation. Our branch-and-price model is exact for integer active-set solutions, but its LP relaxation remains loose. We chose one-sided product constraints for economy: edge variables carry positive objective coefficients, pair variables carry negative coefficients, and we separate triangle cuts only when they are violated. This choice keeps the restricted master small but also explains the long tails in our BP cost table, particularly at larger k and κ. In future work, we will strengthen the relaxation while preserving the lazy active-set design, for example through richer Boolean-quadric inequalities, more systematic triangle-cut separation, and selective addition of the missing McCormick sides when the LP gap is dominated by loose pair variables. The objective remains unchanged; the target is fewer branch-and-bound nodes and LP resolves to certify the same query-centered dense community.

Inverse-out-degree edge weights. Our present objective weighs all directed citation edges equally. In citation networks, however, an edge from a paper with a long bibliography is less specific than an edge from a paper with only a few references. Weighting each outgoing citation by an inverse-out-degree factor would move our density objective closer to a specificity-weighted notion of relatedness: citations from selective papers would count more, while generic survey-style reference lists would contribute less per edge. This modification is particularly relevant to the AvgDeg/BP comparison: AvgDeg favors large regions with many absolute internal edges, while BP favors compact pair density, and inverse-out-degree weighting could sharpen both objectives toward communities supported by more informative citation links.

Warm-start with a densest-subgraph incumbent. BP already benefits from a strong incumbent, since the incumbent controls pruning in every parametric branch-and-bound tree. Our current solver initializes from a BFS seed and greedily removes nodes when doing so improves density or the parametric objective. A stronger warm start would first run a fast densest-subgraph or average-degree heuristic, anchor or repair the result to include q, grow it when needed to satisfy $|S| \geq k$, and pass it to the Dinkelbach outer loop as the initial incumbent. Our experiments motivate this direction directly: AvgDeg is fast and frequently finds high-internal-edge regions, while BP turns such information into compact pair-dense communities, so combining the two could reduce the expensive early iterations of BP.

Semantic information. Our present formulation uses only citation topology. We chose this scope so that the differences among BFS, AvgDeg, and BP could be attributed to graph objectives rather than to text features. For a deployed scholarly-search system, however, papers also carry titles, abstracts, venues, authors, and topic labels. Incorporating such semantic information could improve both retrieval quality and interpretability. One direction is to use semantic similarity as an additional edge weight or as a tie-breaker during frontier pricing and grow-to-k expansion. Another is to report semantic coherence alongside structural density, so that a user can distinguish a graph-dense but topically mixed citation cluster from a graph-dense and semantically coherent research community. We have not pursued these

extensions in this paper, since our central claim is structural; future work should evaluate them carefully so that any added semantic signal complements the structural objective rather than obscures it.

References

[1] Reid Andersen and Kumar Chellapilla. Finding dense subgraphs with size bounds. In *Algorithms and Models for the Web-Graph*, volume 5427 of *Lecture Notes in Computer Science*, pages 25–36. Springer, 2009. doi: 10.1007/978-3-540-95995-3_3.

[2] Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998. doi: 10.1287/opre.46.3.316.

[3] Aleksandar Bojchevski and Stephan Günnemann. Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. In *International Conference on Learning Representations*, 2018.

[4] Francesco Bonchi, David García-Soriano, Atsushi Miyauchi, and Charalampos E. Tsourakakis. Finding densest k-connected subgraphs. *Discrete Applied Mathematics*, 305:34–47, 2021. doi: 10.1016/j.dam.2021.08.032.

[5] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In *Approximation Algorithms for Combinatorial Optimization*, volume 1913 of *Lecture Notes in Computer Science*, pages 84–95. Springer, 2000. doi: 10.1007/3-540-44436-X_10.

[6] Werner Dinkelbach. On nonlinear fractional programming. *Management Science*, 13(7): 492–498, 1967. doi: 10.1287/mnsc.13.7.492.

[7] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[8] Andrew V. Goldberg. Finding a maximum density subgraph. Technical Report UCB/CSD-84-171, University of California, Berkeley, 1984.

[9] Garth P. McCormick. Computability of global solutions to factorable nonconvex programs: Part i–convex underestimating problems. *Mathematical Programming*, 10:147–175, 1976. doi: 10.1007/BF01580665.

[10] Manfred Padberg. The boolean quadric polytope: Some characteristics, facets and relatives. *Mathematical Programming*, 45:139–172, 1989. doi: 10.1007/BF01589101.

[11] Jason Priem, Heather Piwowar, and Richard Orr. Openalex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833*, 2022.

[12] Alexander Veremyev, Oleg A. Prokopyev, Eduardo L. Pasiliao, and Vladimir Boginski. Exact and superpolynomial approximation algorithms for the densest k-subgraph problem. *European Journal of Operational Research*, 262(3):894–903, 2017. doi: 10.1016/j.ejor.2017.04.020.